Kenneth Li, Aspen K Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. 2022. Emergent world representations: Exploring a sequence model trained on a synthetic task. *arXiv preprint arXiv:2210.13382*.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022a. Locating and Editing Factual Associations in GPT. ArXiv:2202.05262 [cs] version: 4.

Kevin Meng, David Bau, Alex J Andonian, and Yonatan Belinkov. 2022b. Locating and editing factual associations in gpt. In *Advances in Neural Information Processing Systems*.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022c. Mass editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.

Jack Merullo, Carsten Eickhoff, and Ellie Pavlick. 2023. Circuit component reuse across tasks in transformer language models.

Tomáš Mikolov, Wen-tau Yih, and Geoffrey Zweig. 2013. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, pages 746–751.

nostalgebraist. 2020. interpreting GPT: the logit lens.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. 2022. In-context learning and induction heads. *Transformer Circuits Thread*. Https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana. Association for Computational Linguistics.

Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. Language Models as Knowledge Bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China. Association for Computational Linguistics.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Elena Voita, Rico Sennrich, and Ivan Titov. 2019. The Bottom-up Evolution of Representations in the Transformer: A Study with Machine Translation and Language Modeling Objectives. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4396–4406, Hong Kong, China. Association for Computational Linguistics.

Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. `https://github.com/kingoflolz/mesh-transformer-jax`.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. 2022. Interpretability in the Wild: a Circuit for Indirect Object Identification in GPT-2 small. ArXiv:2211.00593 [cs].

Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small. In *NeurIPS ML Safety Workshop*.

A Argument-Function Processing in Other Models

In Section 3 we show that GPT2-Medium and Bloom promote the in-context 'argument' token to some function before promoting the answer to that function. In figure 8we show that this effect is present across other models as well in the three

tasks we test. Qualitatively, we find that the pattern is more prominent in models that have more layers, likely because we are able to get more measurements after the FFN updates, so it is less likely that entire argument formation stage happens within a single layer (i.e., after the attention module update – we only take measurements after the FFN update for simplicity). In the extractive task setting, we would not expect the model to go through argument-function processing in order to reach the prediction, since it already appears in context (although this does not preclude it from doing so – it is still a valid way to retrieve the required information). We see that this X shaped pattern disappears when we plot the argument-answer curves for the extractive world capitals data, as shown next to the abstractive setting in Figure 9.

We repeat the random tokens task on GPT-J using the same stimuli as in the main paper to select $\vec{o}$ vectors. We find that we can locate $\vec{o}$ vectors occurring in other models, however the success rate varies for the tasks that we evaluate in this work. Results are shown in Figure 10. Although the uppercasing function works very well, we get weaker responses for the past tense and world capitals mappings. One explanation could be that these tasks are not solved with an as-general solution as in GPT2, but the process for carrying out this intervention depends on hyperparameters which are often model-specific (i.e., the exact layer at which to perform the intervention), so future work is needed to understand where differences between these models lie.

B Additional Results on Ablating FFNs

We include the results for all six models we test for the FFN ablation study for both the colored objects task (Figure 11) and the world capitals task (Figure 12). We find that the trend of abstractive performance dropping off far before extractive performance is reflected across all models.

B.1 +/-o_{case} Intervention on Colors

As described in the main paper, adding o_{case} to the residual stream ($x_{19} + o_{case}$) has the effect of capitalizing the first letter in the word 'brown'. Similar to the results in Sections 2.2 and 2.2, we find that adding o_{case} to the residual stream has the effect of uppercasing the token prediction on arbitrary contextualized representations in the mid layers of GPT2-Medium. However, we also find that lowercasing the first letter can be accomplished by *subtracting* it. Qualitatively, this works much the same way as adding the $\vec{o}$ vectors previously discussed. We show this effect empirically, by showing the difference between replacing the FFN updates in GPT2-Medium with either positive or negative o_case (having the effect of adding or subtracting from the residual stream).

We progressively remove FFNs from the top of the model, and show the effect of adding or subtracting o_{case} in Figure 13. In the abstractive case, we find that accuracy is greatly boosted when adding o_{case} which we identify as implementing an uppercasing function, and reflects the results in Sections 2.2 and 2.2. We find that we can replace the top third of GPT2-Medium FFN layers (FFNs in layers 16-24, around 20% of all parameters) with $+o_{case}$ to gain 25% in total accuracy (from 4.5% to 29.5%) and recovering to 72% of the performance of the un-ablated model (41%). Conversely, if we subtract o_{case} in the abstractive setting to encourage lowercasing (i.e., encouraging the model to output a lowercased answer when the answer it should have a capital first letter), the model immediately hits 0% performance. We see the opposite effect in the extractive setting, where adding $ocase$ hurts performance to a greater degree than subtracting it. According to our results presented so far, we would expect FFNs to be unnecessary for solving the extractive dataset examples, which is possibly why performance is degraded in both cases we intervene, but we don't test this idea in this work.

C What are the Attention Heads Doing?

We focus on the outputs of the FFN layers in this work, but that is not to say that the attention heads are not contributing to the final answer. As shown in Section 5, the attention layers are able to get the final answer when it already appears explicitly in context (when it's extractive). This leads to a possible explanation for why LMs learn to implement argument-function processing. We speculate that this process may be the result of a natural progression in training. When the argument token needs to be transformed ("brown" to "Brown"), the model notices that it is the subject of the next token, and uses attention heads to copy the value of that token into the next token prediction. This operation could be done using mover heads (Wang et al., 2022; Merullo et al., 2023) or induction heads (Olsson et al., 2022). In the following layers, the model

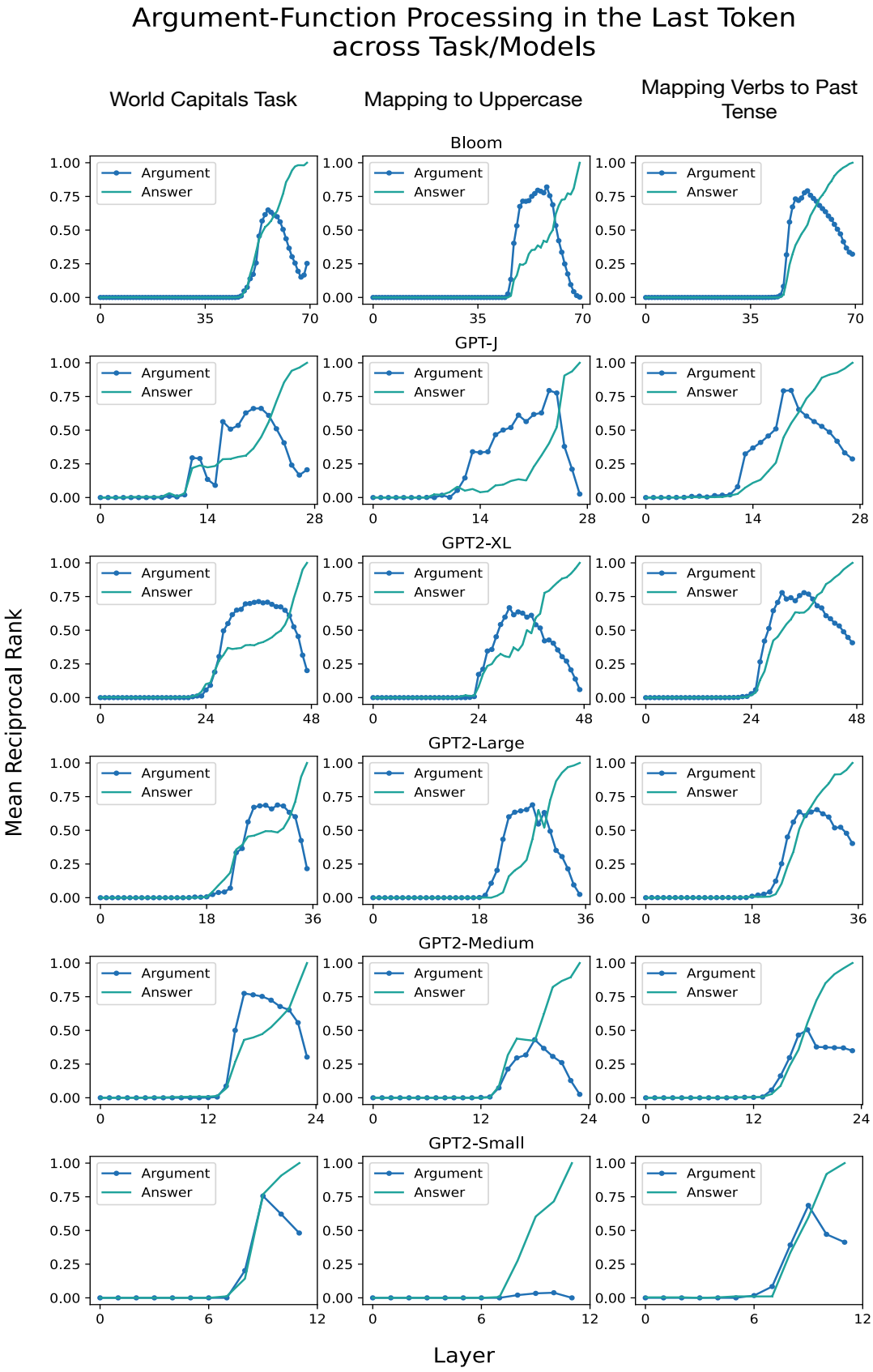

Figure 8: Across several model architectures and tasks, we find evidence that on average, the argument (which appears in context) rises to the top of the vocab distribution before crossing with the answer to the task. We describe this as argument-function processing where the argument to some function is represented in the residual stream before some update from the model is added to it to produce the output of that function. Qualitatively, we observe that models with more layers display this pattern more prominently.